

Разбор решения задачи «Телефонные номера 39.2»

1 Архитектура решения

Алгоритм строит минимальное покрытие телефонного номера словами из словаря с помощью одномерного динамического программирования, двойного полиномиального хэширования и бинарного поиска:

- **Хэширование слов:** Буквы слов переводятся в цифры, после чего вычисляется двойной полиномиальный хэш по модулям $10^9 + 7$ и $10^9 + 9$ с основаниями 313 и 10007, упаковываемый в 64-битное число. Это сводит сравнение строк длины L к сравнению 64-битных чисел за $O(1)$ с вероятностью коллизии $\approx 10^{-18}$.
- **Группировка словаря:** Слова распределяются по векторам в зависимости от их длины ($len \in [1, 100]$), сортируются и очищаются от дубликатов, что позволяет производить поиск за $O(\log M)$ только среди слов нужной длины.
- **Префиксные хэши номера:** Для телефонного номера строятся массивы префиксных хэшей, дающие возможность получать хэш любого фрагмента за $O(1)$.
- **Динамика и восстановление ответа:** Массив $dp[i]$ хранит минимальное число слов для покрытия префикса длины i . Переходы и использованные слова фиксируются в массивах `parent_pos` и `parent_word_id` для последующего восстановления ответа.

2 Детализированная логика основного цикла

Основной цикл алгоритма последовательно перебирает позиционные префиксы `curr_pos` от 1 до `phone_length`, рассматривая текущую позицию как правую границу подстроки. Логическая цепочка шагов на каждой итерации устроена следующим образом:

1. **Итерация по отфильтрованным длинам (`existing_lens`):** Внутренний цикл перебирает только те длины len , для которых в словаре присутствует хотя бы одно слово. Массив `existing_lens` заранее отсортирован по возрастанию.
2. **Раннее прерывание по длине:** Если длина len превышает текущую позицию `curr_pos`, подстрока такой длины заведомо не поместится в рассматриваемом префиксе. Так как длины отсортированы, все последующие значения будут ещё больше, поэтому выполнение внутреннего цикла мгновенно прерывается.
3. **Проверка досягаемости состояния:** Вычисляется позиция начала подстроки $start_pos = curr_pos - len$. Если состояние $start_pos$ недостижимо ($dp[start_pos] = \infty$), приклеивание любого слова к этой позиции не даст валидного покрытия

от начала строки. В этом случае этап вычисления хэша и поиска полностью пропускается.

4. **Вычисление скользящего хэша за $O(1)$:** Если позиция достижима, из префиксного хэша $pref[curr_pos]$ вычитается значение $pref[start_pos] \cdot P^{len}$ по соответствующему модулю. Полученные хэши по обоим модулям объединяются в единое 64-битное число со сдвигом на 32 бита.
5. **Бинарный поиск и релаксация:** Выполняется бинарный поиск найденного хэша в отсортированном векторе слов длины len за $O(\log |words[len]|)$. При совпадении хэшей проверяется условие релаксации: если $dp[start_pos] + 1 < dp[curr_pos]$, новое значение записывается в $dp[curr_pos]$, а в массивы предков фиксируются позиционный индекс $start_pos$ и идентификатор найденного слова.

3 Рекуррентная формула

Состояние $dp[i]$ выражает минимальное количество слов из словаря, необходимое для точного покрытия префикса телефонного номера длины i . Переход в позицию i осуществляется из ранее достижимой позиции $i - l$ (где l — длина слова), если подстрока номера с индекса $i - l$ по $i - 1$ присутствует в словаре среди слов длины l . Значение $dp[i]$ вычисляется как минимальная сумма $dp[i - l] + 1$ по всем подходящим длинам l .

Базовые состояния:

$$dp[0] = 0, \quad dp[i] = \infty \quad (i > 0)$$

4 Полный анализ сложностей

4.1 Временная сложность (Time Complexity)

1. **Считывание и предподсчёт словаря:** Чтение M слов длины L и вычисление их хэшей занимает $O(M \cdot L)$.
2. **Сортировка хэшей словаря:** Сортировка векторов слов по длинам суммарно занимает $\sum O(M_k \log M_k) \leq O(M \log M)$.
3. **Префиксные хэши номера:** Вычисление префиксных хэшей для строки длины N занимает $O(N)$.
4. **Основной цикл DP:** Внешний цикл выполняется N раз, внутренний перебирает K уникальных длин ($K \leq 100$). На каждом шаге поиск выполняется за $O(\log M)$. Итого: $O(N \cdot K \log M)$.
5. **Восстановление ответа:** Проход по массиву предков занимает $O(ans_len) \leq O(N)$.

Итоговая временная сложность: $O(M \cdot L + M \log M + N \cdot K \log M)$

При $N = 100\,000$, $M = 50\,000$, $L = 100$, $K \leq 100$:

$$N \cdot K \log_2(M) \approx 100\,000 \cdot 100 \cdot 15.6 \approx 1.56 \cdot 10^8 \text{ операций,}$$

что выполняется за 0.15–0.3 секунды и существенно ниже лимита в 1.0 с.

4.2 Пространственная сложность (Space Complexity)

- **Исходные слова (dictionary_words):** $M \times (\text{sizeof}(\text{string}) + L) \approx 50\,000 \times 132 \text{ Б} \approx 6.6 \text{ МБ}$.
- **Хэши словаря (words):** $M \times 16 \text{ Б} \approx 0.8 \text{ МБ}$.
- **Массивы DP и предков (dp, parent_pos, parent_word_id):** $3 \times N \times 4 \text{ Б} \approx 1.2 \text{ МБ}$.
- **Префиксные хэши (prefMOD1, prefMOD2):** $2 \times N \times 8 \text{ Б} \approx 1.6 \text{ МБ}$.

Итоговая пространственная сложность: $O(M \cdot L + N)$

Суммарное потребление памяти составляет около **12–15 МБ**, что значительно ниже лимита в **64 МБ**.